

Week 1 Implementation Report

From Raw Text to a Trained LSTM Phishing Classifier

ELMOSSADAK Malak

OUBIHI Lhassan

August 8, 2026

Abstract

This report documents the practical work carried out during Week 1, implemented and run in Google Colab. Starting from raw text cleaning, we built up a full text-classification pipeline step by step: manual preprocessing, tokenization, vocabulary building, sequence padding, and a Keras **Embedding + LSTM + Dense** model. We first validated the pipeline on a toy dataset of six hand-written emails, then scaled it up to a real, public phishing dataset of 10,000 emails from Kaggle, training an LSTM model that reached **94.35% validation accuracy**. The report explains *why* each step is needed, not just *what* the code does, and ends with a critical discussion of the model's current limitations ahead of Week 2.

Contents

1	Objective and Approach	2
2	Step 1 — Manual Text Preprocessing	2
2.1	Lowercasing	2
2.2	Punctuation removal	2
3	Step 2 — Tokenization: From Words to Numbers	3
3.1	Naive tokenization by hand	3
3.2	Tokenization with Keras	3
4	Step 3 — Padding: Fixed-Length Inputs	3
5	Step 4 — Model Architecture	4
5.1	What each layer does	4
6	Step 5 — First Working Prototype (Toy Dataset)	5
7	Step 6 — Scaling Up to a Real Dataset	5
7.1	Data acquisition	5
7.2	Rebuilding the pipeline at scale	6
7.3	Model and training configuration	6
7.4	Results	6
8	Step 7 — Testing on Unseen, Realistic Examples	7
9	Discussion: Current Limitations	7
10	Summary and Next Steps	8

1 Objective and Approach

The goal of Week 1 was to move from the theoretical architecture proposed in the project (Tokenizer → Embedding Layer → LSTM → Sigmoid output) to a working, trained implementation. Rather than jumping directly to the full pipeline, the work was deliberately staged in increasing order of complexity:

1. Clean and tokenize a single sentence by hand, to understand what a tokenizer actually does under the hood.
2. Build a tiny **Embedding + LSTM** model and sanity-check it on 6 synthetic emails.
3. Replace the toy dataset with a real, public Kaggle phishing dataset (10,000 emails) and retrain.
4. Evaluate the trained model on brand-new, unseen sentences.

This staged approach is a standard and sound engineering practice: it isolates bugs in the pipeline (tokenization, padding, model wiring) from bugs in the model itself, before scaling up to real, noisy data.

2 Step 1 — Manual Text Preprocessing

Before any library is used, raw text must be normalized. Two operations were implemented by hand to understand their effect.

2.1 Lowercasing

```
1 text = "URGENT! Click here to verify your bank account now..."
2 clean_text = text.lower()
3 print(clean_text)
```

urgent! click here to verify your bank account now...

Why it matters: without lowercasing, a tokenizer treats "Urgent" and "urgent" as two different words, artificially doubling the vocabulary and diluting the signal the model can learn from either form.

2.2 Punctuation removal

```
1 import string
2 raw_text = "URGENT! Claim your FREE prize by clicking HERE now... !!!"
3 step1 = raw_text.lower()
4 step2 = step1.translate(str.maketrans('', '', string.punctuation))
5 print("No Punctuation:", step2)
```

No Punctuation: urgent claim your free prize by clicking here now

Why it matters: punctuation marks are not informative words; stripping them prevents the vocabulary from being cluttered with tokens like "now..." or "!!!" that would otherwise be treated as distinct, rare words.

3 Step 2 — Tokenization: From Words to Numbers

Neural networks operate on numbers, not strings. *Tokenization* is the process of converting text into a sequence of integer IDs.

3.1 Naive tokenization by hand

```
1 text = "urgent click here to verify account"
2 tokens = text.split()
3 print("Tokens:", tokens)
4 # ['urgent', 'click', 'here', 'to', 'verify', 'account']
```

A manual vocabulary was then built to make the word $\rightarrow$ number mapping explicit:

```
1 word_to_index = {
2     "urgent": 1, "click": 2, "here": 3,
3     "to": 4, "verify": 5, "account": 6
4 }
5 token_ids = [word_to_index[word] for word in tokens]
6 print("Token IDs (Numbers):", token_ids)
```

Token IDs (Numbers): [1, 2, 3, 4, 5, 6]

3.2 Tokenization with Keras

Manually maintaining a word-to-index dictionary does not scale to a real corpus. Keras's `Tokenizer` automates vocabulary building and text-to-sequence conversion:

```
1 from tensorflow.keras.preprocessing.text import Tokenizer
2
3 emails = [
4     "URGENT click here to verify your bank account",
5     "Hello friend how are you doing today"
6 ]
7
8 tokenizer = Tokenizer(num_words=100)
9 tokenizer.fit_on_texts(emails)
10 sequences = tokenizer.texts_to_sequences(emails)
```

Vocabulary: {'urgent': 1, 'click': 2, ..., 'today': 15}
Sequences: [[1, 2, 3, 4, 5, 6, 7, 8], [9, 10, 11, 12, 13, 14, 15]]

Note. `num_words` caps the vocabulary to the N most frequent words; rare words beyond that cap are dropped when converting new text to sequences. This is a deliberate trade-off between vocabulary coverage and model/memory size.

4 Step 3 — Padding: Fixed-Length Inputs

Emails have different lengths, but a neural network expects inputs of a fixed shape. *Padding* solves this by truncating long sequences and filling short ones with zeros up to a chosen length `maxlen`.

```
1 from tensorflow.keras.preprocessing.sequence import pad_sequences
2
```

```

3 sequences = [
4     [1, 2, 3, 4, 5, 6],    # 6 words
5     [7, 8, 9]              # 3 words
6 ]
7 padded_sequences = pad_sequences(sequences, maxlen=6, padding='post')

```

```

[[1 2 3 4 5 6]
 [7 8 9 0 0 0]]

```

`padding='post'` appends zeros at the *end* of short sequences, preserving the original word order at the start — which matters for a recurrent model like an LSTM that processes tokens sequentially.

5 Step 4 — Model Architecture

The classification model follows the architecture proposed in the project plan: an **Embedding** layer, followed by an **LSTM** layer, followed by a single **Dense** sigmoid output neuron.

```

1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import Embedding, LSTM, Dense
3
4 model = Sequential()
5 model.add(Embedding(input_dim=1000, output_dim=16))
6 model.add(LSTM(32))
7 model.add(Dense(1, activation='sigmoid'))
8
9 model.compile(loss='binary_crossentropy', optimizer='adam', metrics=[
    'accuracy'])

```

5.1 What each layer does

- **Embedding(input_dim=1000, output_dim=16)**: maps each of the 1000 possible word IDs to a dense, trainable 16-dimensional vector. Unlike one-hot encoding, this lets semantically related words end up with similar vectors as training progresses.
- **LSTM(32)**: a recurrent layer with 32 hidden units that reads the sequence of word embeddings one token at a time, maintaining an internal state that captures word order and context — something a simple bag-of-words model cannot do.
- **Dense(1, activation='sigmoid')**: compresses the LSTM's final state into a single probability $\in [0, 1]$, interpreted as $P(\text{phishing})$.

Once built, the layers were initialized by fitting on random dummy data for one epoch, then inspected with `model.summary()`:

Layer	Output shape	Parameters
Embedding	(None, 6, 16)	16,000
LSTM	(None, 32)	6,272
Dense	(None, 1)	33
Total trainable parameters		22,305

Table 1: Model summary from the toy 1000-word-vocabulary configuration.

6 Step 5 — First Working Prototype (Toy Dataset)

To validate the full pipeline end-to-end before touching real data, a tiny labeled dataset of six hand-written emails was created and saved to CSV:

```
1 data = {
2     'text': [
3         "Click here to reset your password immediately",
4         "Your account is suspended please verify now",
5         "Hello, are we still meeting for lunch tomorrow?",
6         "Project deadline is next week, please send the report",
7         "Win a free iPhone, click the link to claim!",
8         "Meeting notes from today's scrum session"
9     ],
10    'label': [1, 1, 0, 0, 1, 0]    # 1 = Phishing, 0 = Legitimate
11 }
12 df = pd.DataFrame(data)
```

The same cleaning function (lowercase + remove punctuation) was applied to every row, the cleaned texts were tokenized and padded to `maxlen=6`, and the model was trained for 10 epochs:

```
1 model.fit(X_train, y_train, epochs=10, batch_size=2)
```

```
Epoch 1/10 -- accuracy:  1.0000 -- loss:  0.6537
...
Epoch 10/10 -- accuracy:  1.0000 -- loss:  0.4164
```

Interpretation. 100% training accuracy on 6 examples is expected and *not* meaningful evidence of a good model — with so few examples, the model can easily memorize the dataset. This toy run only confirms that the *pipeline* (cleaning → tokenizing → padding → training) works mechanically, before it is trusted on real data. This is exactly the overfitting risk discussed conceptually in Week 1 theory.

Tested on an unseen sentence, the toy model already showed sensible (if weak) generalization:

```
1 nouveau_test = ["urgent click here to win a prize"]
2 # Score de probabilite (Phishing) : 0.6634 -> flagged as Phishing
```

The trained model and its tokenizer were persisted for reuse:

```
1 model.save("phishing_model.keras")
2 with open('tokenizer.pkl', 'wb') as handle:
3     pickle.dump(tokenizer, handle, protocol=pickle.HIGHEST_PROTOCOL)
```

7 Step 6 — Scaling Up to a Real Dataset

7.1 Data acquisition

The toy dataset of 6 emails is far too small to produce a reliable classifier. The `naserabdullahalam/phishing-email-dataset` was downloaded from Kaggle using the `opendatasets` library, authenticated with a Kaggle API key, and extracted:

```
1 import opendatasets as od
2 dataset_url = 'https://www.kaggle.com/datasets/naserabdullahalam/
3               phishing-email-dataset'
4 od.download(dataset_url)
```

The resulting file, `phishing_email.csv`, contains two columns: `text_combined` (the email body) and `label` (0 = legitimate, 1 = phishing). For Week 1, the first 10,000 rows were used to keep training time reasonable on Colab's free tier.

7.2 Rebuilding the pipeline at scale

The same conceptual pipeline (tokenize → pad → train) was re-applied, with two parameters deliberately increased to match the larger, longer, more diverse real-world text:

```

1 tokenizer = Tokenizer(num_words=10000)           # vocab: 100 ->
   10,000
2 tokenizer.fit_on_texts(texts)
3 sequences = tokenizer.texts_to_sequences(texts)
4
5 X = pad_sequences(sequences, maxlen=100, padding='post') # maxlen:
   6 -> 100
6 y = labels
7
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y, test_size=0.2, random_state=42
10 )

```

Why increase these values? A 6-word cap made sense for short synthetic examples, but real emails run to full sentences and paragraphs; truncating at 6 words would discard almost all of the content. Likewise, a 100-word vocabulary cannot represent the diversity of language across 10,000 real emails. Both parameters were scaled to match the real data distribution.

7.3 Model and training configuration

```

1 model_big = Sequential([
2     Embedding(input_dim=10000, output_dim=32, input_length=100),
3     LSTM(32, return_sequences=False),
4     Dense(1, activation='sigmoid')
5 ])
6 model_big.compile(optimizer='adam', loss='binary_crossentropy',
7     metrics=['accuracy'])
8 model_big.fit(X_train, y_train, epochs=3,
9     validation_data=(X_test, y_test), batch_size=32)

```

Crucially, this run introduces a genuine **held-out validation set** (`X_test`, `y_test`, 20% of the data, unseen during training) — unlike the toy run, this gives a trustworthy estimate of generalization performance.

7.4 Results

Epoch	Train accuracy	Train loss	Validation accuracy
1	80.75%	0.4664	89.35%
2	94.06%	0.1830	76.90%
3	92.06%	0.2418	94.35%

Table 2: Training log for `model_big` on 10,000 real emails (8,000 train / 2,000 validation).

Reading the curve. The dip in validation accuracy at epoch 2 (76.90%, down from 89.35%) despite rising training accuracy is a classic early symptom of overfitting: the model temporarily fit noise specific to the training split. It recovered at epoch 3, but this pattern is worth watching in Week 2 — a validation curve that oscillates like this, over more epochs, would justify adding dropout regularization or reducing model capacity, both mentioned as tools in the original Week 1 theoretical notes.

8 Step 7 — Testing on Unseen, Realistic Examples

The final model, `model_big`, was tested on two brand-new sentences that never appeared in training or validation:

```
1 nouveaux_emails = [
2     "Urgent! Click this link immediately to verify your bank account
3     and win a prize.",
4     "Hi team, let's schedule the project review meeting for next
5     Tuesday."
6 ]
7 sequences_test = tokenizer.texts_to_sequences(nouveaux_emails)
8 X_test_padded = pad_sequences(sequences_test, maxlen=100, padding='
9     post')
10 predictions = model_big.predict(X_test_padded)
```

```
Email: "Urgent! Click this link immediately to verify your bank
account and win a prize."
Phishing probability: [correctly classified as PHISHING]
Email: "Hi team, let's schedule the project review meeting for
next Tuesday."
Phishing probability: [correctly classified as LEGITIMATE]
```

Both predictions matched intuitive human judgement, which is a good qualitative sanity check on top of the 94.35% quantitative validation accuracy.

The scaled-up model and its corresponding tokenizer were saved separately from the toy prototype, to avoid mixing the two vocabularies:

```
1 model_big.save("phishing_model_pro.keras")
2 with open('tokenizer_pro.pkl', 'wb') as handle:
3     pickle.dump(tokenizer, handle, protocol=pickle.HIGHEST_PROTOCOL)
```

9 Discussion: Current Limitations

Being transparent about what this first model does *not* yet do well is as important as reporting what works:

- **Only 10,000 of the full dataset were used.** The full Kaggle dataset is larger; more training data generally improves generalization, at the cost of longer training time.
- **Only 3 epochs were run.** The validation accuracy was still fluctuating (see Section 6.4); more epochs with early stopping would give a more stable, better-justified final model.
- **No regularization (dropout) yet.** Given the oscillation observed at epoch 2, adding dropout between the LSTM and Dense layers is a natural next experiment.

- **No precision/recall/F1 breakdown yet.** Accuracy alone can hide class imbalance issues; a confusion matrix should be computed before treating this model as final.
- **Not yet connected to a web interface.** This is intentionally deferred to Week 2, per the project's phased plan.

10 Summary and Next Steps

Milestone	Status
Manual text cleaning (lowercase, punctuation)	Done
Tokenization (manual, then Keras <code>Tokenizer</code>)	Done
Sequence padding	Done
Embedding + LSTM + Dense model built & compiled	Done
Pipeline validated on toy dataset (6 emails)	Done
Real dataset acquired (Kaggle, 10k emails)	Done
Model retrained on real data with train/val split	Done (94.35% val. accuracy)
Model & tokenizer persisted (<code>.keras</code> , <code>.pkl</code>)	Done
Evaluation metrics (precision/recall/F1)	<i>Planned — Week 2</i>
Dropout / regularization tuning	<i>Planned — Week 2</i>
Flask web interface	<i>Planned — Week 2</i>

Table 3: Week 1 deliverables against the original project plan.

Week 2 will connect this saved model (`phishing_model_pro.keras` + `tokenizer_pro.pkl`) to a Flask route, so that a user-submitted email can be tokenized, padded, and scored using exactly the pipeline validated here, before the final web interface described in the original project proposal is built.